universidade de aveiro
departamento de eletrónica, telecomunicações e informática

Software Architectures

Architecture Checkpoint Report

Assignment 2: Architectural Evolution of nopCommerce

Group 107

Afonso Ferreira	113480
Tomás Brás	112665
Hugo Ribeiro	113402
Rodrigo Abreu	113626

April 30, 2026

Contents

Chapter 1

Introduction and Scenario

1.1 Scenario Choice

This project follows **Scenario C — Omnichannel Commerce Core**. VerdeMart Retail started by using nopCommerce as its online storefront, but the business now needs more than a web shop. It wants web sales, warehouse execution, shipping, store operations, support, and customer visibility to work together as part of one commerce experience.

In this scenario, nopCommerce becomes the commerce core of that wider ecosystem. A web order should be reflected in operational systems such as warehouse and shipping, and changes that happen outside nopCommerce, such as stock or fulfillment updates, should become visible again in the customer and operator experience. This is important because those surrounding systems may be slow, stale, unavailable, or inconsistent, but the commerce experience still needs to remain useful and understandable.

1.2 Business Context and Drivers

VerdeMart Retail began as a single-channel online shop. Three business pressures now demand an architectural response:

1. **Operational growth.** Fulfillment has moved to a dedicated warehouse, and physical stores now handle pickup and return flows. nopCommerce has no first-class model for communicating with either system, so operators manage these flows manually or through fragile workarounds.
2. **Stock inconsistency.** The web catalog shows stock numbers from nopCommerce's local model. Warehouse and store sales can reduce physical stock without immediately updating the online view. This creates overselling risk and damages customer trust.
3. **Operational fragility.** Any slow or unavailable external system currently causes visible degradation during checkout or shipment processing. There is no durable retry path, no degraded-state visibility, and no recovery workflow.

These pressures translate directly into the quality attribute scenarios defined in Chapter 4: availability during warehouse or shipping failure, consistency under stale inventory, performance under campaign load, recoverability after provider outages, and modifiability as new operational channels are added.

1.3 Surrounding Systems in Scope

The assignment permits the use of simulators or stubs for surrounding systems. For this project, the following systems from the Scenario C ecosystem are represented:

- **Warehouse execution system** (OpenBoxes or equivalent WMS) — receives fulfillment requests, returns picked, packed, delayed, and failed acknowledgements.
- **Shipping provider** (WireMock simulator) — handles label creation and tracking updates; can be made unavailable to test the recoverability scenario.
- **Store operations / POS** — publishes in-store sale events and pickup state changes that must update the online inventory view.
- **ERP / back-office** (ERPNext or equivalent) — receives order and financial summary events from the commerce core.

RabbitMQ is used as the message broker for asynchronous propagation, consistent with the Scenario C surrounding system list.

1.4 Core Architectural Problem

Currently, nopCommerce mainly acts as the online shop: it owns the storefront, catalog browsing, cart, checkout, customer accounts, and order creation inside one web-centred platform. This works while the business is mostly operating through the online channel, but it does not address an omnichannel environment where warehouse execution, shipping, store operations, and support systems also influence the customer experience.

The core architectural problem is deciding how this online shop can evolve into a commerce core without assuming that every surrounding operational system is always fast, available, and consistent. The architecture must define what remains inside nopCommerce, what moves around it, and how the system behaves when warehouse or shipping state is delayed, stale, unavailable, or later recovered.

Chapter 2

Current-State Analysis

2.1 Architecture Overview

nopCommerce follows an onion architecture, where all dependencies point inward toward the core. The solution is split into four main layers: `Nop.Core`, `Nop.Data`, `Nop.Services`, and the presentation layer (`Nop.Web` and `Nop.Web.Framework`).

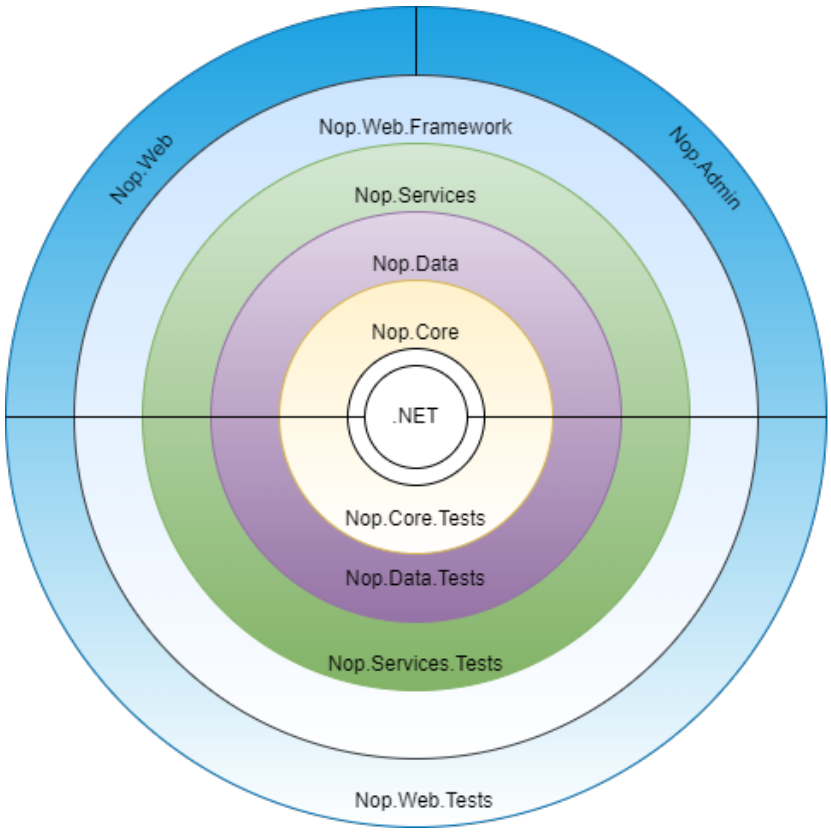

Figure 2.1: Current layered/onion architecture of nopCommerce.

Each layer has a specific role:

- **`Nop.Core`**: The innermost layer. Contains domain entities, caching, events, and helper classes shared across the whole solution. It has no dependencies on any other project. Key Scenario C entities live here: `Order`, `OrderItem`, `Product`, `ProductWarehouseInventory`, `StockQuantityHistory`, `Warehouse`, and `Shipment`.
- **`Nop.Data`**: Separates data-access logic from business objects. nopCommerce uses a Linq2DB

Code-First approach, with FluentMigrator for schema migrations and Fluent API for persistence mappings.

- `Nop.Services`: The outermost layer of the Application Core. It holds business logic, validations, and calculations, and uses a repository pattern to expose features to layers above it. This is where order processing, inventory adjustment, shipping, and checkout logic live. Key services for Scenario C are `OrderProcessingService`, `IShipmentService`, `IProductService`, and `IStockQuantityHistoryService`.
- `Nop.Web` and `Nop.Web.Framework`: The presentation layer. `Nop.Web` contains controllers, models, views, and the storefront and admin area. `Nop.Web.Framework` is a shared class library with common presentation logic used by both the public site and the admin panel. Controllers reference only services, never repositories directly, which provides a clean seam for adding integration boundaries without touching the presentation layer.
- **Plugins**: Loaded as separate Visual Studio projects and extend nopCommerce for payments, shipping, authentication, search, and other integrations. They run inside the same runtime as the monolith and share its process lifecycle.

2.2 How a Purchase Works Today

A customer order flows from the storefront controllers in `Nop.Web` down through `Nop.Services`. `OrderProcessingService` creates the order, adjusts inventory, persists state via repositories, and publishes internal events like `OrderPlacedEvent`. Shipment updates follow later through shipment services, which update order status and trigger notification events.

This works cleanly when nopCommerce owns the full picture. It breaks down when an external warehouse, POS, or carrier system owns part of the order's lifecycle and may be slow, unavailable, or return conflicting data.

The internal event system publishes events in-process to registered consumers and awaits each one. If a consumer fails, the error is logged and execution continues. There is no message broker and no standard durable integration retry, replay, or dead-letter mechanism. This is fine for cache invalidation or plugin reactions — it is not sufficient for reliable integration with external systems.

2.3 Scenario C Readiness Assessment

The table below classifies each Scenario C-relevant capability as ready to use, ready but requiring extension, or missing and needing to be built. This assessment drives the scope of the target architecture defined in subsequent chapters.

Capability	Status	Gap or action needed
Order creation and customer checkout (`OrderProcessingService`)	Ready	Extend with outbox record written in the same transaction.
Local stock and warehouse model (`ProductWarehouseInventory`, `StockQuantityHistory`)	Extend	Add source, freshness, stale, and conflict metadata fields (defined in target model, Chapter 6).
Shipment entity and tracking (`Shipment`, shipping plugins)	Extend	Add explicit integration status fields: sent to WMS, warehouse delayed, awaiting label.
Admin order and shipment visibility	Extend	Surface integration status, retry count, and dead-letter indicator alongside existing order screens.
Scheduled task infrastructure	Extend	Useful for an initial prototype or fallback execution host; the target architecture uses a dedicated Integration Worker for publishing and retry scheduling.
Durable outbox for integration tasks	Missing	New table in nopCommerce DB, written atomically with the order; polled by Integration Worker.
Integration status and retry state	Missing	New tables: outbox records, retry attempts, dead-letter queue, correlation log.
Inventory freshness and conflict detection	Missing	New metadata fields and conflict-detection logic in the Inventory Adapter.
Warehouse and shipping adapters	Missing	New independently deployable processes consuming integration events from RabbitMQ.
Support and operations visibility view	Missing	New read model exposing pending, degraded, failed, and recovered work to operators.

The five "Ready" or "Extend" capabilities confirm that nopCommerce is a strong starting point for the evolution. The five missing capabilities define the scope of the integration layer and adapter work introduced in the target architecture. Synchronous plugin calls that currently expose checkout to external failures are replaced by asynchronous adapter communication, and the absent failure-state and reconciliation model is addressed by the outbox, retry, and dead-letter design recorded in ADRs 2, 6, and 7.

Chapter 3

Domain and Bounded Contexts

3.1 Domain Overview

Scenario C changes the role of nopCommerce from an online storefront into the commerce core of VerdeMart's wider omnichannel ecosystem. The domain is therefore not limited to product browsing and online checkout. It also includes fulfillment coordination, inventory visibility, store activity, shipping progress, support visibility, and recovery when surrounding systems are delayed, stale, unavailable, or contradictory.

The current nopCommerce baseline already contains important domain concepts for this scenario. `Nop.Core` defines entities such as `Order`, `OrderItem`, `Product`, `Warehouse`, and `Shipment`. It also includes warehouse inventory and stock history entities such as `ProductWarehouse Inventory` and `StockQuantityHistory`. `Nop.Services` contains the application logic around order placement, inventory adjustment, shipping, plugins, internal events, and scheduled tasks. `OrderProcessingService` persists the order, moves cart items into order items, adjusts inventory, sends notifications, and publishes an in-process `OrderPlacedEvent`. These are useful seams, but they are not yet a durable integration boundary for external warehouse, POS, shipping, or support systems.

The target model keeps nopCommerce as the owner of the customer-facing commercial transaction. It does not decompose the monolith into many services. Instead, it defines explicit boundaries around the operational capabilities that surround the commerce core. Warehouse execution, store/POS stock changes, shipping-provider state, and support operations visibility must communicate through APIs, events, or adapters. They must not read or write the nopCommerce database directly.

The main domain tension is ownership under imperfect information. nopCommerce can own the canonical web order, but it cannot assume that every external stock number, warehouse acknowledgement, or carrier update is immediately correct. The bounded contexts below make uncertain external state explicit so the system remains useful and explainable when surrounding systems are delayed or unavailable.

3.2 Relevant Subdomains

Subdomain	Type	Description
Customer commerce	Core	Product discovery, cart, checkout, payment result handling, customer order history, and the canonical web order.
Fulfillment coordination	Core / supporting	Propagates accepted orders to warehouse, store, ERP, and shipping capabilities through a durable target-state integration boundary.
Inventory availability	Supporting	Presents customer-facing availability using nopCommerce stock data and projected updates from warehouse and store/POS systems, including freshness and conflict state.
Warehouse fulfillment	External supporting	Owns picking, packing, dispatch preparation, fulfillment acknowledgement, and warehouse failure reasons.
Store operations / POS	External supporting	Owns in-store sale events, pickup activity, and local stock changes that must become visible again in the commerce experience.
Shipping	External generic	Owns label creation, carrier tracking updates, and carrier-specific failure responses.
Support and operations visibility	Supporting	Gives support agents and operators a view of cross-system work that is pending, failed, recovered, or needs attention.

3.3 Bounded Contexts

3.3.1 nopCommerce Commerce Core Context

This context remains inside nopCommerce. It owns catalog presentation, customer accounts, shopping carts, checkout, payment result handling, order creation, and customer-facing order history. This maps naturally to the existing layered/onion structure described in the current state analysis.

For Scenario C, this context remains the system of record for the canonical web order. It may publish an order-accepted integration task in the target architecture, but it should not depend synchronously on warehouse, POS, or shipping providers during checkout. This supports availability and performance by keeping the customer request path separate from slow operational dependencies.

Owns:

- customer accounts, carts, checkout state, and payment result state;
- canonical web orders and order items;
- customer-facing order history and local order status;
- local catalog and stock concepts already represented by `Product`, warehouse inventory records, and stock quantity history.

Does not own:

- warehouse picking and packing execution;
- POS sales ledgers or store stock ledgers;
- carrier labels and carrier tracking source of truth;

- durable external-integration work and operational recovery records.

Boundary rule: external systems may receive commands or events derived from nopCommerce orders, but they must not access nopCommerce tables directly.

3.3.2 Integration Coordination Context

This is a target-state context introduced around nopCommerce, not a complete capability in the current baseline. Its responsibility is to make communication with surrounding systems durable, observable, and recoverable. It is the main place where the architecture records delivery state, correlation identifiers, idempotency keys, retry attempts, timeouts, dead-letter entries, and reconciliation work.

The current in-process event publisher is useful inside the monolith, but it is not enough for reliable external integration because consumers run in the same process and failures are logged rather than retained as durable work. Scheduled tasks remain a useful execution seam for polling, retries, timeout detection, or cleanup, but they need explicit integration state to operate safely.

Owns:

- target-state integration messages, outbox-style tasks, and delivery status;
- correlation identifiers linking a web order to warehouse, shipping, inventory, and support records;
- idempotency keys to prevent duplicate fulfillment or label creation during retries;
- retry, dead-letter, timeout, and reconciliation state.

Does not own:

- the canonical web order;
- warehouse operational execution;
- carrier tracking source of truth;
- POS sales.

Boundary rule: every external integration must pass through an adapter, API, or event contract. Shared database access is not an allowed shortcut.

3.3.3 Warehouse Fulfillment Context

This context represents the warehouse execution capability in Scenario C. It owns physical fulfillment state such as accepted for picking, picked, packed, delayed, failed, ready for dispatch, and warehouse acknowledgement. nopCommerce already has `Warehouse`, warehouse inventory, and shipment-related concepts, but the current baseline does not model a separate WMS lifecycle such as “sent to warehouse”, “warehouse delayed”, or “fulfillment rejected”.

In the target architecture, nopCommerce sends a fulfillment request asynchronously after order acceptance. The commerce core then receives a projection of warehouse state for customer, admin, and support visibility. This supports availability and recoverability because a slow or unavailable warehouse does not block checkout.

Owns:

- picking, packing, dispatch preparation, and warehouse failure reasons;
- warehouse acknowledgement or rejection of fulfillment work;
- warehouse-side timing and operational exceptions.

Typical target-state messages:

- `FulfillmentRequested`;
- `FulfillmentAccepted`;
- `OrderPicked` or `OrderPacked`;
- `FulfillmentDelayed` or `FulfillmentFailed`.

3.3.4 Inventory Availability Context

This context provides cross-channel stock visibility for the web storefront, checkout, operators, and support agents. The baseline nopCommerce product model includes stock quantity, multiple-warehouse inventory records, reserved quantity, and stock history. These records are a strong starting point, but they do not distinguish fresh external stock from stale external stock, nor do they record contradictory source-system claims as first-class business states.

The target context treats inventory as a projection that includes source and freshness metadata. Warehouse and store/POS systems may report changes at different times. The web storefront may therefore show stock that is temporarily stale. Instead of hiding this as a technical problem, the model exposes states such as `InventoryStale` and `ReconciliationRequired`. This supports consistency by making inconsistency visible and manageable rather than pretending all channels are immediately consistent.

Owns:

- customer-facing availability projections;
- source, last-confirmed time, stale-state, and conflict-state metadata in the target model;
- inventory discrepancy and reconciliation records.

Does not own:

- every physical stock movement in the warehouse or store;
- internal POS sale accounting;
- the canonical web order.

3.3.5 Shipping Context

This context represents shipping-provider or carrier integration. The baseline already has `Shipment`, tracking number, shipped date, delivered date, ready-for-pickup date, shipping plugins, and admin shipment operations. These support the target architecture, but they do not by themselves provide a durable label-request workflow or provider-outage handling.

In the target architecture, label creation and tracking updates are handled through a shipping adapter or provider API. nopCommerce may display shipping progress as a projection, while the carrier remains the owner of carrier-specific tracking state.

Owns:

- carrier label creation and label failures;
- carrier tracking updates;
- provider-specific shipping errors.

Typical target-state messages: `ShipmentReadyForLabel`, `LabelCreationRequested`, `LabelCreated`, `LabelCreationFailed`, and `TrackingUpdated`.

3.3.6 Store Operations / POS Context

This context represents physical store activity. nopCommerce already supports pickup-in-store concepts through order and shipment fields and the pickup plugin, but a full POS system remains outside the current baseline. In Scenario C, the POS/store context owns in-store sale events, store stock changes, pickup readiness, and pickup completion.

The important boundary is that the POS does not update nopCommerce inventory tables directly. It publishes or exposes store events through an adapter. The inventory availability context can then update the commerce-facing projection, and the support context can expose any issue that needs operational attention.

Owns:

- POS sale events and local store stock changes;
- pickup readiness and pickup completion events;
- store-side operational exceptions.

Does not own:

- online checkout;
- canonical web order creation;
- nopCommerce database tables.

3.3.7 Support and Operations Context

This context gives operators and support agents visibility into omnichannel state. The current nopCommerce admin area already exposes orders, shipments, warehouses, stock history, plugins, and scheduled tasks. The target architecture extends this with operational read models for cross-boundary work and its current outcome.

This context matters because a resilient integration is still operationally weak if people cannot explain what happened to an order or stock change. Degraded states are therefore part of the business model, not only log messages.

Owns:

- operational status views and support-facing explanations;
- failure visibility, recovery history, and corrective actions;
- audit trail of cross-boundary integration attempts in the target model.

Does not own:

- warehouse, store, or shipping execution;
- the canonical web order;
- external stock or carrier ledgers.

3.4 Boundary Model

Boundary	Direction	Interaction style	Reason
Customer/admin to nopCommerce	Inbound	Synchronous	Browsing, checkout, and order visibility require immediate responses.
nopCommerce to integration coordination	Outbound	Asynchronous target-state event or task	Order propagation must survive warehouse or shipping delay without blocking checkout.
Integration coordination to warehouse adapter	Outbound	Async command/API call with retry	Fulfillment requests may be delayed or unavailable and must remain recoverable.
Warehouse adapter to integration coordination	Inbound	Event/callback/poll result	Picked, packed, delayed, failed, or accepted state must become visible.
Store/POS to inventory adapter	Inbound	Event/API update	Store stock changes should update availability without direct database coupling.
Integration coordination to shipping adapter	Outbound	Async command/API call with retry	Label creation and carrier calls can fail and must be idempotent.
Integration coordination to support view	Internal	Read model/projection	Operators need visibility into work that is waiting, failed, recovered, or requires action.

3.5 Boundary Rules

1. **nopCommerce owns the canonical web order.** External systems may receive order commands or events, but they do not become the source of truth for the web order.
2. **No external system reads or writes the nopCommerce database directly.** This keeps the monolith's database internal and avoids hidden coupling across service boundaries.
3. **Checkout does not synchronously depend on warehouse, POS, or shipping.** External work is recorded and processed asynchronously in the target architecture.
4. **External operational state is projected back into the commerce experience.** Fulfillment status, shipping status, and stock availability can be shown to customers, administrators, or support agents, but they are projections with source and freshness metadata where needed.
5. **Every cross-boundary message needs correlation and idempotency.** This prevents duplicate fulfillment or duplicate label creation when retries happen after timeout, failure, or uncertain acknowledgement.
6. **Degraded states are domain states.** States such as pending warehouse confirmation, warehouse delay, awaiting shipping label, stale inventory, and reconciliation required should be visible to the appropriate customer, support, or operator view rather than hidden only in logs.
7. **Adapters translate external language into internal language.** Warehouse, POS, ERP, and carrier terms should not leak directly into the core order model.

3.6 Architecture Diagram

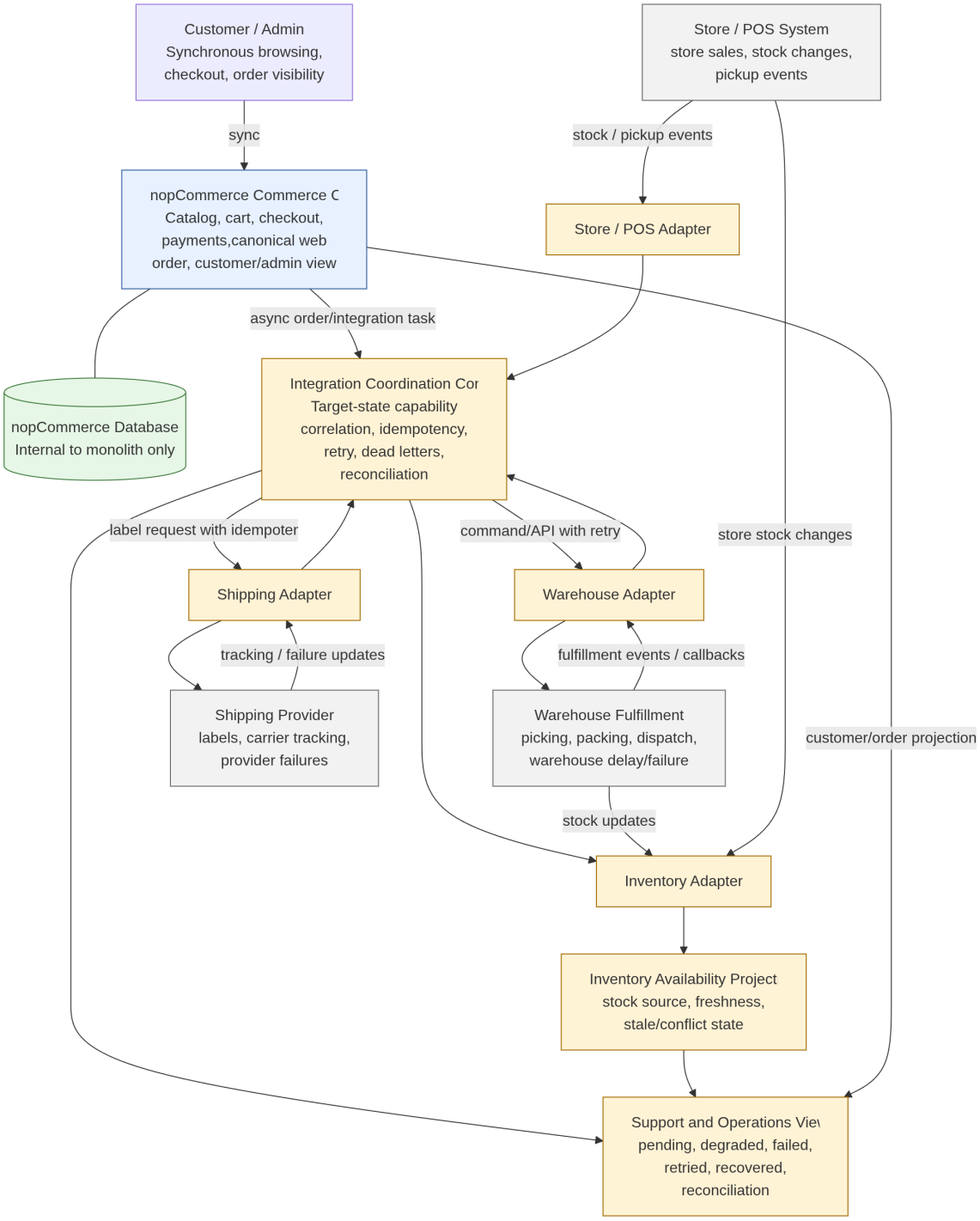

Figure 3.1: Domain and boundary model for Scenario C.

This model is traceable to the quality attributes in Chapter 4. Availability is supported by accepting and persisting web orders without synchronously requiring the warehouse or shipping provider. Consistency is handled by explicit stale and conflicting inventory states rather than by assuming immediate agreement across channels. Performance is protected by keeping slow integration work outside the checkout request path. Recoverability is supported by retry, idempotency, correlation, and reconciliation in the target integration context. Modifiability is

supported by adapters and by forbidding shared database access across external boundaries.

Chapter 4

Quality Attribute Scenarios

Quality attribute scenarios make the architectural drivers measurable. For the VerdeMart omnichannel scenario, the most important pressure is not only that nopCommerce must create orders correctly, but that the commerce experience must remain understandable while warehouse, shipping, store, and support systems are slow, stale, unavailable, or later recovering.

The scenarios below use the six-part structure from the course material: stimulus source, stimulus, artifact, environment, response, and response measure.

4.1 Architectural Drivers

The scenarios are driven by five architectural concerns:

- **Availability:** web orders must not be lost when warehouse or shipping systems fail.
- **Consistency:** inventory and order state may be temporarily stale or contradictory, but inconsistencies must be visible and recoverable.
- **Performance:** customer-facing operations must stay responsive during backend synchronization and sales peaks.
- **Recoverability:** failed integration work must be retained, retried, and resolved without operator intervention where possible.
- **Modifiability:** new operational channels must be integrated without rewriting checkout or sharing the nopCommerce database.

4.2 Scenario 1 - Availability under Warehouse Failure

Driver: Availability and operational continuity.

Part	VerdeMart scenario
Stimulus source	Warehouse execution system
Stimulus	The warehouse API takes more than 10 seconds to respond or rejects fulfillment requests for newly paid web orders.
Artifact	Fulfillment synchronization and the order status view shown to customers and operators
Environment	Friday afternoon peak traffic, with 3 times normal order volume, while the warehouse dependency is degraded but the web shop is still accepting orders
Response	The order is accepted and persisted locally; the fulfillment request is stored in the outbox for asynchronous retry; the order is shown as awaiting warehouse confirmation; subsequent orders are not blocked by the degraded dependency.
Response measure	No paid order is lost; no duplicate fulfillment request is sent on retry (idempotency key check); pending fulfillment state is visible to support within 30 seconds; the first retry attempt starts within 5 minutes.

4.3 Scenario 2 - Consistency under Stale Inventory

Driver: Consistency and recoverable inventory state.

Part	VerdeMart scenario
Stimulus source	Store point-of-sale system
Stimulus	A store sale reduces stock by 5 units, but the update reaches nopCommerce 3 minutes after the web catalog still showed those units as available.
Artifact	Inventory availability view used by product pages, cart validation, and checkout
Environment	Concurrent web and physical-store sales for the same product during a promotion, with external stock updates delayed by up to 3 minutes
Response	The commerce core detects that the stock record has exceeded its staleness threshold, marks it stale, revalidates stock before checkout completion, and records an inventory discrepancy for reconciliation.
Response measure	Product pages show stale stock status within 30 seconds of the staleness threshold being exceeded; overselling stays under 1% of affected order lines; inventory discrepancies are queued for reconciliation within 1 minute.

4.4 Scenario 3 - Performance during Omnichannel Order Bursts

Driver: Performance under customer-facing load.

Part	VerdeMart scenario
Stimulus source	Web customers, store operators, and customer support agents
Stimulus	A campaign creates 5 times normal traffic for product browsing, cart validation, order creation, and order-status queries.
Artifact	Storefront, checkout path, order status view, and integration event publishing path
Environment	Peak campaign period, with 5 times normal traffic and background synchronization still running
Response	The system prioritizes checkout writes, serves order-status reads from a local projection, and processes warehouse and shipping synchronization asynchronously without blocking the customer request path.
Response measure	Checkout p95 latency remains below 2 seconds; order-status read p95 remains below 1 second; at least 99% of integration events are published or queued within 60 seconds.

4.5 Scenario 4 - Recoverability after Shipping Outage

Driver: Recoverability after external dependency failure.

Part	VerdeMart scenario
Stimulus source	Shipping provider
Stimulus	Shipment label creation fails for 50 ready-to-dispatch orders and the provider later becomes available again.
Artifact	Shipping integration workflow, retry queue, and order shipment status
Environment	Warehouse has packed orders, but the external shipping provider is unavailable for 20 minutes
Response	The system stores failed label requests in the retry queue, keeps orders in a clear awaiting-shipping state, and resumes label creation automatically when the provider recovers without requiring operator re-submission.
Response measure	100% of failed label requests are retained in the retry queue; operators see the outage state within 30 seconds; 95% of queued labels are created within 10 minutes after provider recovery.

4.6 Scenario 5 - Modifiability for New Operational Channels

Driver: Modifiability and controlled integration growth.

Part	VerdeMart scenario
Stimulus source	VerdeMart business team and integration developers
Stimulus	A new store-operations system must receive order-ready-for-pickup events and send pickup-completed updates back to the commerce core.
Artifact	Integration layer, domain events, and order status update boundary
Environment	Existing web, warehouse, and shipping integrations are already in production
Response	The architecture allows the new channel to be added through a new adapter and event subscription without changing checkout logic or sharing the nopCommerce database with the external system.
Response measure	The new integration requires changes in no more than one adapter and one application-service boundary; no direct external database access is introduced; existing integration flows for warehouse and shipping are unaffected.

4.7 Scenario 6 — Inventory Reconciliation after Cross-Channel Conflict

Driver: Consistency under contradictory external state.

Part	VerdeMart scenario
Stimulus source	POS system and warehouse management system
Stimulus	POS reports 8 units in stock; WMS reports 3 units for the same SKU. The difference exceeds the configured tolerance of 2 units and persists for more than 5 minutes.
Artifact	Inventory Availability context, reconciliation record, and operator view in the Support and Operations area
Environment	Concurrent web and in-store activity for a promoted product; both systems are online but reporting contradictory quantities
Response	The system detects the discrepancy, sets `ConflictFlag` on the affected SKU, limits checkout commits for that SKU to the lower reported quantity, and creates an operator-visible reconciliation task.
Response measure	Conflict is detected and flagged within 30 seconds of the divergence exceeding the tolerance; checkout is bounded by the lower reported quantity; the reconciliation task is visible to operators within 1 minute; if both sources agree within 30 minutes the flag is cleared automatically, otherwise operator action is required.

4.8 Traceability to Architectural Drivers and Decisions

Scenario	**Architectural driver**	**Addressed by**
Availability under warehouse failure	Availability, recoverability	ADR 1 (async fulfillment), ADR 2 (outbox)
Consistency under stale inventory	Consistency	ADR 3 (adapter boundaries), inventory freshness metadata
Performance during omnichannel order bursts	Performance	ADR 1 (async path), ADR 4 (monolith retained)
Recoverability after shipping outage	Recoverability, availability	ADR 2 (outbox), ADR 6 (retry policy), ADR 7 (dead-letter)
Modifiability for new operational channels	Modifiability	ADR 3 (adapter, no shared DB)
Inventory reconciliation after conflict	Consistency	ADR 3 (adapter boundaries), inventory conflict and reconciliation model

Chapter 5

Architectural Approach

5.1 Chosen Framework

The selected architectural design framework is **Attribute-Driven Design (ADD)**.

ADD fits this project because the VerdeMart scenario is mainly driven by measurable quality attribute pressures. The main question is not only which components should exist, but how the commerce core should behave when surrounding operational systems are delayed, stale, unavailable, or recovering.

In this project, ADD is applied as follows:

1. Drivers identified: availability, consistency, performance, recoverability, and modifiability.
2. Six quality attribute scenarios defined, each with a measurable response.
3. Tactics selected per scenario: asynchronous integration, transactional outbox-style event publishing, read-model separation, retry queue, reconciliation, adapter pattern, and observability.
4. Target architecture decomposed around those tactics: nopCommerce Core, Integration Layer, Operational Adapters, External Systems, and Support and Operations View.
5. Validation focused on the riskiest path: asynchronous propagation, degraded dependency handling, retry, and recovery.

5.2 Justification

The chosen scenario requires nopCommerce to evolve from a web storefront into the commerce core of a wider operational ecosystem. This creates architectural pressure in five main areas: availability, consistency, performance, recoverability, and modifiability.

ADD is appropriate because each of these pressures can be expressed as a concrete quality attribute scenario. Warehouse failure drives the need for asynchronous fulfillment and retry. Stale inventory drives the need for inventory visibility, revalidation, and reconciliation. Order bursts drive the need to protect customer-facing paths from slow backend synchronization. Shipping outage drives the need for durable retry and explicit recovery state. New operational channels drive the need for adapters and stable integration boundaries.

This gives a direct trace from the problem to the architecture:

ADD input	Scenario	Architectural consequence
Availability	Scenario 1	Avoid synchronous dependency on warehouse systems during checkout.
Consistency	Scenarios 2, 6	Introduce explicit inventory state, stale-state visibility, conflict detection, and reconciliation.
Performance	Scenario 3	Keep checkout and order reads separate from slow integration processing.
Recoverability	Scenario 4	Store failed integration work durably and retry after dependency recovery.
Modifiability	Scenario 5	Add external systems through adapters instead of direct database coupling.

5.3 Why Not ACDM or ADM as the Main Framework

ACDM was considered because it is useful for architectural decision-making, risk discussion, and stakeholder validation. However, it is not the main framework for this project because the strongest design input is the set of measurable quality attribute scenarios. The project must show runtime behavior under failure, delay, stale state, and recovery; ADD gives a more direct path from those scenarios to concrete architectural tactics and component responsibilities.

ACDM-style reasoning is still useful later in the project, especially when documenting ADRs and explaining rejected alternatives. For example, the decision to use asynchronous integration can be recorded as an ADR with alternatives such as direct synchronous calls or shared database access. However, the decision itself is primarily driven by the availability and recoverability scenarios, which makes ADD the better primary method.

ADM/TOGAF was also considered, but it is broader than necessary for this assignment. ADM is better suited to enterprise-wide transformation across business, data, application, and technology architecture. VerdeMart does have an omnichannel business context, but the scope of this work is a focused software architecture evolution of nopCommerce, not a full enterprise architecture programme.

5.4 Tactics Selected per Quality Attribute

ADD requires explicit tactics to address each quality attribute scenario. The following table records which tactics were chosen and why each was preferred over the alternatives considered in the ADRs.

Quality attribute	Tactics selected	Architectural manifestation
Availability	Asynchronous integration, durable recording of work	Outbox pattern (ADR 2): order and its integration task are committed atomically; warehouse unavailability does not block checkout.
Consistency	Stale-state marking, explicit reconciliation	Inventory metadata fields (`IsStale`, `ConflictFlag`); reconciliation record created on divergence; checkout revalidates against `PendingReconciliation`.
Performance	Separate synchronous and asynchronous paths, read-model projection	Customer request path contains no external I/O; integration work runs outside the HTTP pipeline in the Integration Worker.
Recoverability	Idempotency, retry with backoff, dead-letter handling	Each integration task carries a correlation identifier and idempotency key; the retry scheduler applies exponential backoff; failed work beyond the retry limit is moved to a dead-letter queue visible to operators.
Modifiability	Adapter pattern, stable event contracts, no shared database	Each external system is reached through a dedicated adapter (ADR 3); internal message contracts are owned by the Integration Coordination context; no adapter accesses nopCommerce tables directly.

5.5 How ADD Shapes the Target Architecture

Using ADD means that each architectural element in the target design should answer a specific quality attribute pressure. The surrounding systems are not added only because they exist in the business scenario; they are added because they create design pressure that nopCommerce must handle explicitly.

The target architecture emphasises asynchronous integration because Scenario 1 requires that a warehouse timeout does not block checkout. A direct synchronous call from checkout to the warehouse cannot satisfy that constraint, so accepted orders are propagated through a durable integration path.

Status is made explicit for pending, degraded, failed, and recovered work because Scenarios 1 and 4 require support agents and operators to understand what is happening during warehouse or shipping failure. Retry and reconciliation are part of the architecture because Scenarios 2, 4, and 6 assume stale inventory, contradictory cross-channel stock, failed label creation, and later recovery. External systems are connected through adapters because Scenario 5 requires new operational channels to be added without rewriting checkout or sharing the nopCommerce database.

The next chapter uses these ADD-driven tactics to define the target architecture and its component responsibilities.

Chapter 6

Target Architecture

6.1 Overview

The target architecture keeps nopCommerce as the commerce core, but stops treating every surrounding operational capability as a direct synchronous dependency. nopCommerce remains responsible for the customer-facing commerce experience: catalog browsing, cart, checkout, customer accounts, payment result handling, and order creation. Around it, the architecture adds integration boundaries for warehouse, inventory, shipping, store operations, and support visibility.

The main architectural change is the introduction of an integration layer between nopCommerce and the surrounding systems. This layer exposes adapters for external systems, publishes and consumes integration events, stores work that cannot be completed immediately, and retries or reconciles failed operations. This allows the commerce core to remain useful when warehouse, shipping, or store systems are slow, stale, unavailable, or recovering.

The architecture is therefore not a full microservice rewrite of nopCommerce. It is an evolutionary architecture where the monolith keeps the responsibilities that are already stable inside nopCommerce, while volatile omnichannel coordination is moved to explicit integration boundaries.

6.2 Main Components and Services

Component	Responsibilities	Communicates with
nopCommerce Core	Catalog, cart, checkout, payment result handling, order creation, customer accounts, canonical web order	Customers and admin (HTTP); Integration Worker via outbox
Integration Worker	Reads outbox, publishes commands and events to RabbitMQ, processes inbound adapter events, runs retry and dead-letter scheduler	nopCommerce DB (outbox tables); RabbitMQ queues
Warehouse Adapter	Translates fulfillment commands into WMS API calls; receives picked, packed, delayed, and failed fulfillment events; relays status back	RabbitMQ; WMS API
Inventory Adapter	Normalizes stock changes from warehouse and POS; detects stale and conflicting records; emits availability and reconciliation events	RabbitMQ; WMS/POS stock event sources
Shipping Adapter	Requests carrier labels; stores failed label requests in the retry queue; retries on provider recovery; relays tracking updates	RabbitMQ; Shipping provider API
Store Operations Adapter	Receives in-store sale events, pickup readiness, and pickup-completed updates; relays store stock changes to the inventory adapter	RabbitMQ; POS system
Support and Operations View	Exposes cross-boundary integration state to operators and support agents; surfaces failure reasons, retry counts, correlation identifiers, and reconciliation tasks	Integration metadata DB (read model)

The Support and Operations View deserves particular attention because a resilient integration layer is operationally weak if support agents cannot explain what happened to an order. The view exposes:

- orders in *pending*, *retrying*, *failed*, and *dead-lettered* states, with the current retry count and the scheduled time for the next retry attempt;
- the specific failure reason recorded by the adapter — for example, warehouse timeout, label provider HTTP error, or inventory conflict;
- correlation identifiers that link a web order to its warehouse, shipping, inventory, and POS integration records, making it possible to trace a single order across all systems;
- a re-queue action that moves a dead-lettered record back to the retry queue without requiring direct database access or operator scripting;
- inventory records with `IsStale` or `ConflictFlag` set, together with the associated reconciliation task status and whether operator action is still required.

6.3 Synchronous and Asynchronous Interactions

The architecture uses synchronous interactions only when the user needs an immediate answer and the dependency is inside the commerce core. Product browsing, cart operations, checkout validation, payment result handling, and order creation remain synchronous from the customer's point of view. These operations must be fast and predictable because they directly affect the buying experience.

Interactions with warehouse, shipping, inventory synchronization, and store operations are primarily asynchronous. After an order is created, nopCommerce writes an integration task to the outbox atomically with the order record. The Integration Worker reads the outbox and publishes a fulfillment command to RabbitMQ. The Warehouse Adapter consumes the command from its dedicated queue and forwards it to the WMS API. Because messages accumulate in RabbitMQ while an adapter is unavailable, the Integration Worker and the adapter processes are decoupled: if the warehouse is slow or its adapter is restarting, the order is not lost and the checkout path is not blocked; the fulfillment state becomes pending or degraded and is retried later.

Inventory updates also follow an asynchronous model. Warehouse and store systems publish stock changes, and the Inventory Adapter normalizes them into integration events. The Integration Worker then updates the availability projection used by nopCommerce. Because this data may be stale, checkout still performs stock validation before completion, and detected conflicts are sent to reconciliation.

Shipping follows the same pattern. When the warehouse marks an order as ready to dispatch, the shipping adapter requests the label. If the provider fails, the label request is stored, the order is marked as awaiting shipping, and automatic retry resumes when the provider recovers.

6.4 Data Ownership

Data ownership is explicit to avoid hidden coupling between systems. nopCommerce owns customer accounts, carts, checkout state, payment result state, and the canonical web order. External systems are not allowed to read or write the nopCommerce database directly.

The warehouse system owns physical fulfillment state, such as picking, packing, dispatch preparation, and fulfillment failures. The shipping provider owns carrier label generation and carrier tracking updates. Store operations own in-store events such as pickup handling and point-of-sale stock changes. The integration layer owns integration metadata: pending work, retry attempts, correlation identifiers, failed messages, and reconciliation records.

nopCommerce may keep local views of external state, such as order fulfillment status, inventory availability, or shipping status, but these are projections used to support customer and operator visibility. They are not shared databases and should be updated through integration messages or adapter APIs.

Inventory projections extended in the target model carry additional metadata per stock record: `SourceSystem` (which external system last reported the value), `LastConfirmedUtc` (when the value was last confirmed), `IsStale` (set when the record exceeds its per-source freshness threshold), `ConflictFlag` (set when two sources disagree beyond a configured tolerance), and `PendingReconciliation` (set when a discrepancy is queued for resolution). These fields make inconsistent external state visible rather than silently propagating stale numbers to customers.

6.5 Cross-Cutting Concerns

Reliability, observability, consistency, security, and modifiability are addressed by the decisions recorded in the ADRs rather than handled ad hoc. The transactional outbox (ADR 2) and idempotency keys (ADR 5) make integration durable and duplicate-safe: every fulfillment command and label request carries a correlation identifier and an idempotency key, so retries after timeout or uncertain acknowledgement cannot create duplicate warehouse picks or double labels. The retry policy with exponential backoff and circuit breaker (ADR 6) and the dead-letter queue with operator intervention model (ADR 7) ensure that degraded state is retained and visible rather than silently discarded. The adapter pattern with no shared database (ADR 3) enforces both security and modifiability: external systems interact only through adapters, APIs, or RabbitMQ events with controlled permissions, and a new POS, warehouse system, or shipping

provider can be added by implementing or replacing one adapter without touching checkout logic or the nopCommerce data model.

6.6 Integration Layer Deployment Model

The Integration Layer is a separately deployable .NET service or worker process. It is not embedded inside the nopCommerce web application, which means it can be restarted, scaled, or replaced without touching the commerce core. It shares the nopCommerce database only to read and write the outbox table and the integration-metadata tables; it does not access catalog, customer, or order tables directly.

The deployment model for this evolution has three units:

1. **nopCommerce web application** — the customer-facing storefront and admin area. Writes orders and outbox records atomically. Exposes APIs consumed by adapters through its existing Web API plugin.

2. **Integration Worker** — a background service that reads the outbox, publishes messages to RabbitMQ, processes inbound events from adapters, updates integration-status projections, and runs the retry and dead-letter scheduler.

3. **Surrogate adapter services** — lightweight services or WireMock stubs representing the warehouse, shipping provider, POS, and ERP. Each one has its own network identity so timeouts and outages can be simulated independently.

The nopCommerce database is the shared persistence layer for the commerce core and the Integration Worker. This is acceptable because the outbox and metadata tables have a clear owner (the Integration Coordination context) and are not exposed to external adapters.

6.7 Main Architecture Diagram

The diagram is organized into five zones, each corresponding to a data-ownership boundary described in Section 6.4.

Zone 1 — nopCommerce Commerce Core. The top zone shows the customer-facing synchronous path: browser or mobile client sends HTTP requests to the nopCommerce web application, which processes cart, checkout, and payment result handling entirely within the monolith. The order and its outbox record are written in a single database transaction (ADR 2) before the HTTP response is returned. No external system is called during this path, so warehouse or shipping unavailability cannot block checkout.

Zone 2 — Integration Worker and message broker. The middle integration zone shows the Integration Worker reading the outbox and publishing commands and events to RabbitMQ. RabbitMQ is the durable transport between the Integration Worker and all adapter processes. Arrows entering this zone from the right represent acknowledgements, fulfillment updates, stock changes, and tracking events arriving from adapters. The Integration Worker applies the retry policy (ADR 6), routes failures to the dead-letter queue (ADR 7), and writes integration-status projections consumed by the Support and Operations View.

Zone 3 — Adapter services. The adapter zone contains four independently deployable adapter processes: Warehouse Adapter, Inventory Adapter, Shipping Adapter, and Store Operations Adapter. Each adapter has its own RabbitMQ queue and its own outbound connection to the corresponding external system. The dashed boundaries in this zone indicate that each adapter can be stopped, replaced, or restarted without affecting the others or the commerce core.

Zone 4 — External systems. The external-systems zone represents the systems that VerdeMart does not own: the WMS (OpenBoxes or equivalent), the shipping carrier API, the

POS system, and the ERP. These systems communicate exclusively through their adapter; no arrow crosses directly from an external system into the nopCommerce zone.

Zone 5 — Support and Operations View. The bottom zone shows the read model populated by the Integration Worker. Operators and support agents query this view to inspect pending, retrying, failed, and dead-lettered work without accessing the nopCommerce database or the RabbitMQ management interface directly.

The diagram uses two arrow styles to distinguish the two interaction modes. Solid arrows indicate synchronous HTTP calls, which are confined to the customer path and the nopCommerce internal layers. Dashed or open arrows indicate asynchronous message flows through RabbitMQ, which cover fulfillment commands, fulfillment updates, inventory changes, label requests, and tracking updates. This visual separation maps directly to Scenario 1 (warehouse unavailability must not block checkout) and Scenario 3 (customer path latency must not be affected by backend synchronization).

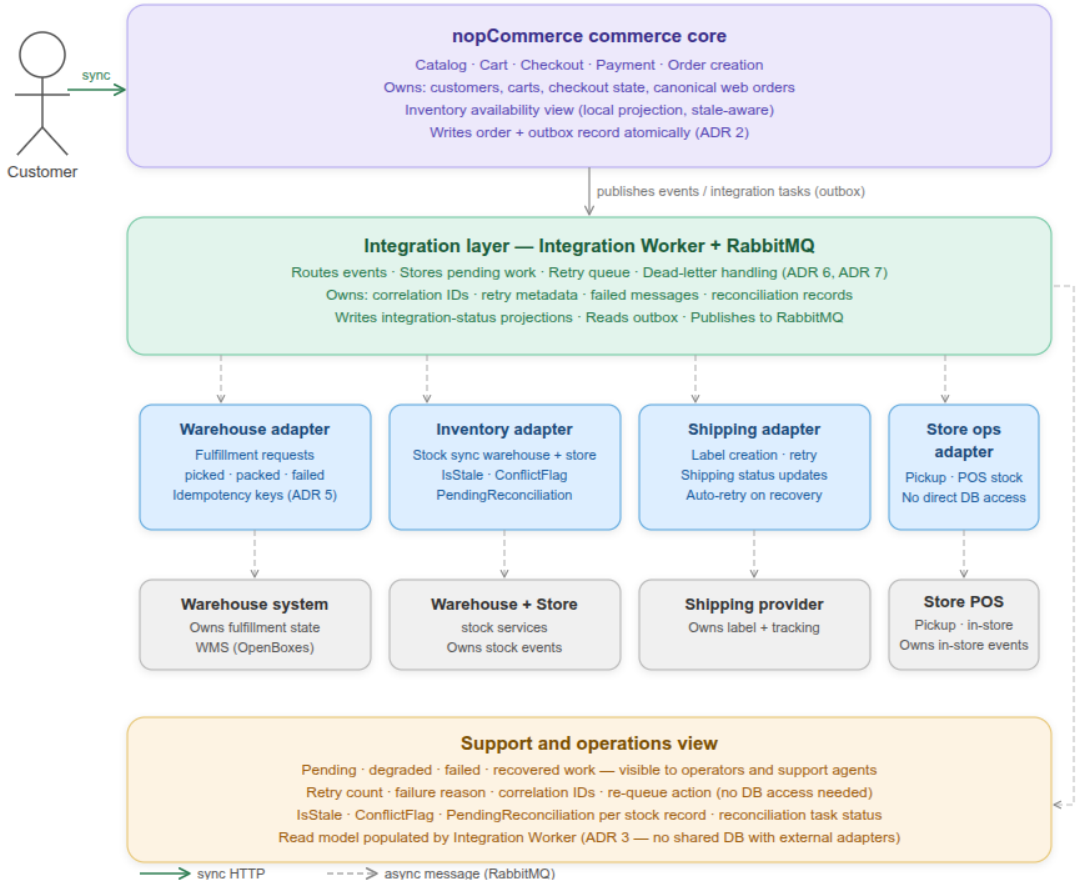

Figure 6.1: Target architecture for Scenario C: nopCommerce as an omnichannel commerce core.

6.8 Runtime Interaction: Warehouse Failure and Recovery

The static architecture diagram shows which components exist and how they are connected. This section describes what happens at runtime when QA Scenario 1 is exercised: a customer places an order while the warehouse system is unavailable.

Step 1 — Checkout and atomic commit. The customer submits the checkout form. `OrderProcessingService` validates the cart, processes payment, creates the order, and writes one outbox record representing `FulfillmentRequested` in the same database transaction. The

HTTP response is returned to the customer as soon as the transaction commits. At this point, the warehouse system has not been contacted and its availability is irrelevant to the customer.

Step 2 — Integration Worker picks up the outbox record. On its next polling cycle, the Integration Worker reads the unpublished outbox record, assigns a correlation identifier and idempotency key, and publishes a `FulfillmentRequested` command to the RabbitMQ exchange dedicated to warehouse work. The outbox record is marked as sent.

Step 3 — Warehouse Adapter attempts delivery and fails. The Warehouse Adapter consumes the command from its queue and calls the WMS API. The WMS is unavailable and the call times out. The Adapter reports failure to the Integration Worker, which marks the outbox record as retrying and schedules the next attempt according to the exponential backoff policy defined in ADR 6. The Support and Operations View is updated to show the order in a pending warehouse confirmation state.

Step 4 — Retry loop and circuit breaker. The Integration Worker retries the delivery at increasing intervals. If five consecutive attempts from the Warehouse Adapter all fail, the circuit breaker opens and blocks further attempts for the cooldown period. An alert is sent to the operations team. The order remains visible in the Support and Operations View with its current retry count and the scheduled time for the next attempt.

Step 5 — Warehouse recovers. When the WMS becomes available again, the circuit breaker probe attempt succeeds, the circuit closes, and the retry loop resumes. The Warehouse Adapter delivers the `FulfillmentRequested` command successfully. The WMS acknowledges the request, and the Adapter publishes a `FulfillmentAccepted` event back to the Integration Worker.

Step 6 — Integration Worker records recovery. The Integration Worker receives the acknowledgement, marks the outbox record as accepted, and updates the order's integration status projection. The Support and Operations View transitions the order from retrying to accepted. The idempotency key stored on the outbox record ensures that any duplicate delivery caused by uncertain acknowledgement during the retry loop does not create a second fulfillment request at the warehouse.

Figure 6.2 shows this sequence. Steps 1 and 2 happen within the customer request latency budget. Steps 3 through 6 happen entirely outside the customer request path, which is why warehouse unavailability cannot block or degrade the checkout experience.

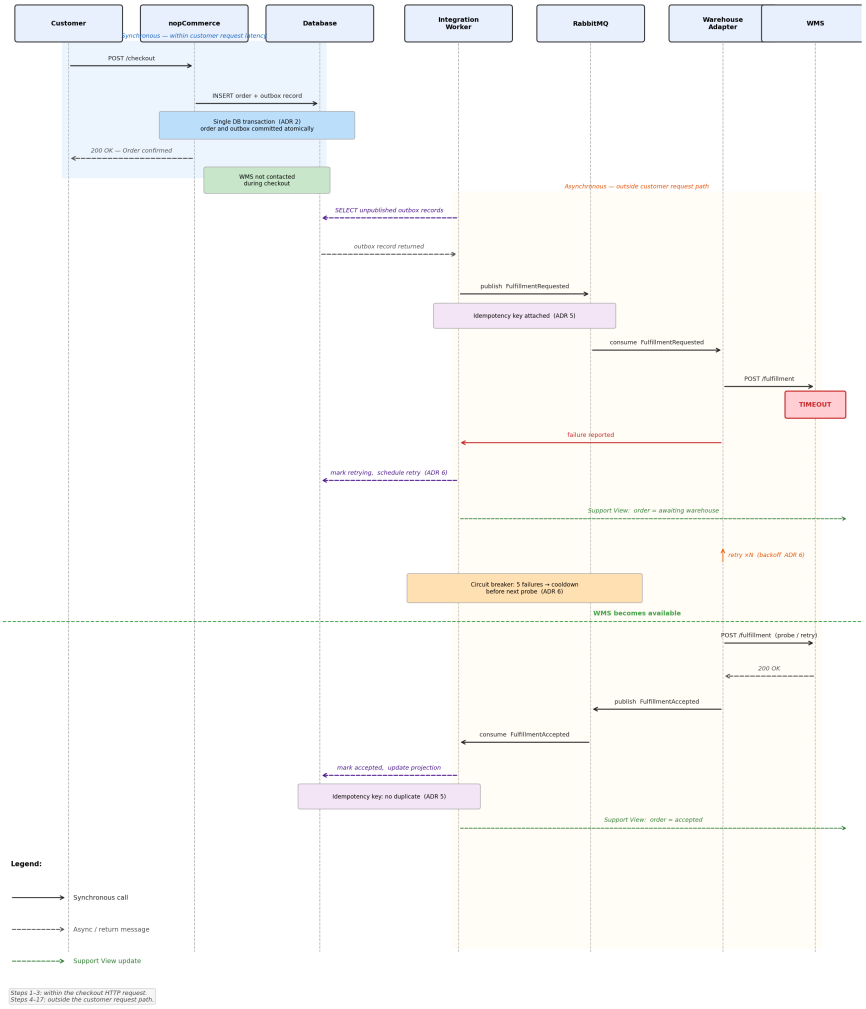

Figure 6.2: Runtime sequence for QA Scenario 1: order placement under warehouse failure and subsequent recovery.

Chapter 7

Architectural Decisions

This chapter records the major architectural decisions made during the evolution of nopCommerce for the VerdeMart omnichannel scenario. Each decision is traceable to one or more quality attribute scenarios from Chapter 4 and to the bounded contexts defined in Chapter 3. Rejected alternatives are included with explicit reasons, since the value of a decision is inseparable from understanding what was considered and set aside.

7.1 ADR 1 — Asynchronous Fulfillment Propagation

7.1.1 Context

After a web order is accepted and paid, nopCommerce must notify the warehouse execution system. In the current baseline, external communication happens either through synchronous plugin calls or in-process events, both of which tie the checkout path directly to the availability of the downstream system.

QA Scenario 1 (Availability under Warehouse Failure) requires that a warehouse timeout or rejection does not prevent a web order from being recorded. Scenario 4 (Recoverability after Shipping Outage) further requires that failed fulfillment and shipping work is retained and retried, not silently dropped. A synchronous warehouse call from within the checkout flow cannot satisfy either constraint.

7.1.2 Decision

Fulfillment propagation is handled asynchronously. When nopCommerce accepts an order, it records an integration task for the fulfillment request. This task is picked up outside the customer request path, so a slow or unavailable warehouse has no effect on checkout completion. If the warehouse rejects or fails to acknowledge the request, the task stays pending and is retried by the Integration Coordination context.

Shipping follows the same model. Once the warehouse marks an order as ready to dispatch, the shipping adapter requests the carrier label independently. Provider unavailability results in a stored, retryable request rather than a visible checkout failure.

This is consistent with the boundary rule established in Chapter 3: checkout does not synchronously depend on warehouse, POS, or shipping systems.

7.1.3 Alternatives Considered

Synchronous API call to the warehouse during checkout. Calling the warehouse directly after order creation is straightforward to implement, but it makes checkout latency and success rate dependent on warehouse response times. During peak traffic, a warehouse response exceeding

ten seconds degrades the buying experience, and an outage blocks all order confirmations entirely. Scenario 1 rules this out.

In-process event bus (current nopCommerce approach). The existing `OrderPlacedEvent` fires in-process and awaits registered consumers. It works well for internal reactions like cache invalidation or plugin hooks, but it offers no durability for external integration. A process restart between order commit and event dispatch loses the event with no retry path. Chapter 2 identifies this limitation explicitly as a gap for the target scenario.

7.1.4 Consequences

Checkout becomes independent of warehouse and shipping availability, which is the primary goal. The trade-off is that fulfillment state is eventually consistent: the order model must expose intermediate states such as awaiting warehouse confirmation or fulfillment delayed, and those states must be visible to support agents and operators. Integration tasks also need idempotency guarantees to prevent duplicate fulfillment commands when retries occur, which is the subject of ADR 5.

7.2 ADR 2 — Transactional Outbox for Durable Integration Task Publishing

7.2.1 Context

ADR 1 requires that fulfillment work is propagated asynchronously, but that alone does not solve durability. A common approach publishes a message after the database transaction commits. The problem is the gap between the two operations: if the process restarts or the broker is unreachable at that moment, the integration task is lost while the order appears confirmed to the customer. The warehouse never receives the request, and without explicit detection, nobody knows.

The Integration Coordination context defined in Chapter 3 owns integration messages, correlation identifiers, idempotency keys, and retry state. For this ownership to be meaningful, there must be a reliable way to record integration work at the point the order is created.

7.2.2 Decision

Integration tasks are written in the same database transaction as the order. When `OrderProcessingService` creates an order, it also inserts an outbox record in the same transaction. The Integration Worker periodically reads unpublished records, dispatches them to the integration layer, and marks them published. If the process restarts before the worker runs, the record survives and is processed on the next cycle.

This gives a simple atomicity guarantee: the order and its integration task either both exist or neither does. At-least-once delivery follows naturally, which means consumers must be idempotent because duplicate messages can arrive during retries after uncertain acknowledgements.

7.2.3 Alternatives Considered

Publish after commit (dual-write). Committing the order and then publishing to the broker is the path of least resistance. The gap it creates is not theoretical: a network blip, broker restart, or process crash at that moment silently loses the integration task. Recovering requires reconciling the order table against warehouse acknowledgements by hand, which is fragile and hard to sustain operationally.

Scheduled task polling without an outbox table. A task could scan for orders in a specific status and attempt delivery without a dedicated outbox record. This sounds simpler but

effectively reconstructs outbox semantics informally. Tracking which orders have been sent and ensuring the delivery attempt and the status update are atomic requires the same care as an explicit outbox, but without the clear ownership and structure.

Relying on broker-side durability guarantees alone. Some brokers provide strong delivery guarantees once a message is accepted. The difficulty is that the producer still needs to successfully reach the broker within the same logical operation as the order commit. When the broker is temporarily unavailable at order creation time, the task is never enqueued. An outbox moves the broker dependency to the Integration Worker, which tolerates broker unavailability at commit time without affecting the customer.

7.2.4 Consequences

No paid order is silently dropped due to a transient broker or process failure. The worker polling interval introduces a small propagation delay between order creation and warehouse notification, which is acceptable given the asynchronous design. The outbox table becomes integration-critical infrastructure and must be covered by database backups and operational monitoring. The publishing interval also determines the minimum propagation latency, which must remain within the bounds set by QA Scenario 1.

7.3 ADR 3 — Adapter Pattern with No Shared Database Across External Boundaries

7.3.1 Context

VerdeMart's operational ecosystem includes at minimum a warehouse execution system and a shipping provider, with POS and ERP systems also present. As the business grows, further operational systems may be added or replaced. Each one has its own protocols, data models, and failure behaviour.

The nopCommerce database is a single schema that mixes catalog, orders, customers, inventory, shipments, configuration, and logs. Allowing external systems to query or write this schema directly would create implicit dependencies on internal table structures, making it difficult to evolve either side independently. It also creates ambiguous data ownership: if the warehouse system writes directly to the nopCommerce stock table, it is unclear which system is the authoritative source and which is a consumer.

QA Scenario 5 (Modifiability for New Operational Channels) requires that a new system can be integrated through no more than one adapter boundary, without touching checkout logic or coupling to the nopCommerce schema.

7.3.2 Decision

Each external system is connected through a dedicated adapter that translates between the internal integration contract and the system's specific protocol, data model, and error conventions. The Warehouse, Inventory, Shipping, and Store Operations adapters are each independently replaceable. State flowing back into nopCommerce, fulfillment status, stock updates, carrier progress, arrives through events or adapter APIs and is stored as a projection with source and freshness metadata. No external system reads or writes the nopCommerce database directly.

7.3.3 Alternatives Considered

Direct database access for external systems. Giving the warehouse system or ERP read or write access to the nopCommerce database is the lowest-effort path for a small and stable set of integrations. The problem is that it couples every external system to the internal schema. Any

table rename, column change, or migration that makes sense for the commerce core becomes a cross-team coordination problem. It also makes data ownership opaque, which complicates debugging and reconciliation when systems disagree.

A single generic integration service. Routing all external events through one integration service reduces the number of deployable components. The cost is that a single service must handle the distinct failure modes, retry policies, and domain translations for every external system simultaneously. A shipping provider outage and a warehouse delay are different problems requiring different handling; merging them into one service makes both harder to reason about and monitor independently.

Plugin calls from within the nopCommerce process. nopCommerce plugins can call external systems, and this pattern is already used for payment and shipping-rate providers. For the omnichannel integration boundaries in Scenario C, however, plugins inherit the same limitations as synchronous calls: they block the request path, they have no built-in retry or dead-letter mechanism, and their availability is tied to the monolith process. They remain the right tool for internal extensibility, not for durable external coordination.

7.3.4 Consequences

Each external system has a bounded, explicit integration contract. Replacing a warehouse provider means replacing its adapter; the commerce core and the other adapters are unaffected. New channels follow the same pattern: one new adapter, one set of event subscriptions, no changes to checkout. The adapter boundary also enforces the language separation noted in Chapter 3: warehouse, POS, and carrier terminology does not leak into the core order model. The cost is that every new system needs an adapter implementation, but the scope of that work is predictable and isolated.

7.4 ADR 4 — nopCommerce Monolith Retained as the Commerce Core

7.4.1 Context

When an omnichannel scenario adds warehouse, POS, shipping, and ERP integrations, a natural design instinct is to decompose the central platform into smaller services. nopCommerce already separates concerns into layers and the plugin model gives it modularity at the code level, so the question of whether to extract services from it is worth taking seriously.

The quality attribute scenarios in Chapter 4 identify five pressures: availability, consistency, performance, recoverability, and modifiability. Looking at each one, none of them requires extracting checkout, order management, inventory, or catalog from the monolith. The existing layered architecture already provides clear seams. Breaking those capabilities into separate services would introduce distributed transaction problems and network dependencies that do not currently exist, without resolving any of the five identified pressures.

7.4.2 Decision

nopCommerce remains the commerce core and is not decomposed. Checkout, catalog, payments, customer accounts, and canonical order management stay inside the monolith. The main new independently deployable platform component introduced by this evolution is the Integration Coordination context, which owns the outbox, retry, idempotency, and reconciliation behaviour described in ADR 2. Adapters for warehouse, shipping, inventory, and store operations sit around this layer. The monolith interacts with the Integration Coordination context through its outbox table and a publishing worker, remaining decoupled from the internal design of any individual adapter.

7.4.3 Alternatives Considered

Extract order management as an independent service. Separating order creation, payment result handling, and order history into a standalone service would produce an independent deployment unit, but it removes a large body of stable, well-tested functionality from the monolith. More importantly, it would introduce a synchronous dependency between the storefront and the new order service at the point of checkout, which directly conflicts with the availability requirement in QA Scenario 1. The decomposition adds complexity without improving any of the measured quality attributes.

Extract inventory as an independent service. Given QA Scenario 2 (Consistency under Stale Inventory), a dedicated inventory service was considered. The scenario requires cross-channel stock visibility with freshness metadata and stale-state detection. The Inventory Adapter already provides this by updating projections inside nopCommerce through integration messages, extending the existing stock model rather than replacing it. Extracting inventory into a separate service would require network calls for every stock check at checkout, a new database to own the projection, and a service boundary that the adapter model renders unnecessary.

Full microservices rewrite. Decomposing nopCommerce into catalog, cart, checkout, payment, order, inventory, and notification services would require recreating a large volume of working functionality across independently operated services, coordinating distributed transactions across service boundaries, and shifting most of the project effort from architectural evolution to infrastructure. The result would be a more complex system with the same functional scope.

7.4.4 Consequences

The monolith retains its operational simplicity: one deployable unit for the full customer-facing commerce experience, with all existing capabilities, checkout, payments, catalog, admin, and plugin extensibility intact. The Integration Coordination context is tightly bounded to reliability and coordination, while adapter services remain boundary-specific and replaceable. The main limitation is that the commerce core cannot be scaled independently from its own checkout and order-management data model, but this is sufficient for the load levels identified in QA Scenario 3 and is not a constraint that justifies decomposition at this stage. This decision does not preclude future extraction of read-heavy services such as the catalog or order-status projection if load patterns justify it later.

7.5 ADR 5 — Idempotency Key Strategy for Integration Messages

7.5.1 Context

ADR 1 establishes asynchronous fulfillment, and ADR 2 guarantees at-least-once delivery through the outbox. At-least-once delivery means that the same fulfillment or label-creation message may be delivered more than once during retries. If the warehouse or shipping adapter processes the same message twice, the result can be a duplicate fulfillment request or a second shipping label for the same order — both of which cause operational problems.

7.5.2 Decision

Every integration message carries an `IdempotencyKey` generated at message creation time as a UUID. The key is stored in the outbox record so it is stable across retries. Each adapter is responsible for detecting a duplicate key and returning a success acknowledgement without repeating the side-effecting operation. The Integration Coordination context records key-to-outcome pairs for a configurable deduplication window (default: 48 hours). Successful outcomes

are cached to prevent unnecessary processing.

7.5.3 Alternatives Considered

Order-ID-based idempotency. Using the nopCommerce order identifier as the idempotency key is simpler, but it conflates the order identity with the delivery attempt identity. If an order requires two separate fulfillment requests for different warehouse splits, both would carry the same key and the second would be incorrectly treated as a duplicate.

Idempotency enforced only at the database level. Some adapters add a unique constraint on the order identifier in their local database. This prevents duplicate processing but ties the idempotency guarantee to adapter internals rather than the message contract. It cannot be relied upon for external systems where the adapter is a surrogate or stub.

7.5.4 Consequences

Adapters must implement idempotency checks on the `IdempotencyKey`. The deduplication window creates a bounded storage requirement for the key-outcome table. The risk of missing a duplicate outside the 48-hour window is low for the use case, since retries for any single message should complete well within that period.

7.6 ADR 6 — Retry Policy and Circuit Breaker for Adapter Calls

7.6.1 Context

When a warehouse or shipping adapter is unavailable, the Integration Worker retries the failed message. Without a controlled retry policy, the worker can overwhelm a recovering adapter with a burst of accumulated retries, turning a temporary outage into a sustained overload. A circuit breaker pattern can prevent this by stopping retry attempts after a threshold is reached and resuming only after a health check succeeds.

7.6.2 Decision

The retry policy applies exponential backoff with the following parameters: initial delay of 2 seconds, backoff multiplier of 2, maximum delay of 5 minutes, and maximum of 10 retry attempts. After 5 consecutive failures from the same adapter, the circuit breaker opens and blocks further attempts for a configurable cooldown period (default: 5 minutes). After the cooldown, one probe attempt is made. If it succeeds, the circuit closes and normal retry resumes. If it fails, the cooldown restarts.

Messages that exhaust all retry attempts are moved to the dead-letter queue described in ADR 7.

7.6.3 Alternatives Considered

Fixed-interval retry without a circuit breaker. Fixed-interval retries are simpler but can create a thundering-herd effect when a dependency recovers. They also provide no protection against sustained outages: a dependency down for 30 minutes would receive hundreds of retry attempts per message before the outbox is drained.

Linear backoff. Linear backoff reduces the thundering herd compared to fixed intervals but converges to a high retry rate more quickly than exponential backoff and does not provide the same degree of protection during long outages.

7.6.4 Consequences

The maximum propagation delay for a fulfilled order under sustained warehouse failure is bounded by the backoff ceiling and max retries (approximately 50 minutes of active retry before dead-letter). The circuit breaker protects the recovering adapter from burst load. Operators must be informed when a circuit opens, because it means fulfillment work has stopped flowing to the affected adapter. This requires a monitoring alert and a visible circuit-state indicator in the Support and Operations View.

7.7 ADR 7 — Dead-Letter Queue and Operator Intervention Model

7.7.1 Context

After the maximum retry attempts defined in ADR 6 are exhausted, the integration message has failed permanently in the automated path. A decision must be made about what happens next: the message can be silently discarded, moved to an archive, or presented to an operator for manual action.

Discarding messages is not acceptable for fulfillment or shipping work, since a lost fulfillment message means a paid order is never sent to the warehouse. The architecture therefore needs a durable dead-letter record and a defined escalation process.

7.7.2 Decision

Failed messages are written to a dead-letter table in the nopCommerce database with the following fields: the original message payload, the idempotency key, the correlation identifier (order ID), the adapter that failed, the failure reason for the last attempt, the timestamp of the first and last attempt, and the escalation state (new, acknowledged, resolved).

The Support and Operations View surfaces dead-letter records alongside pending and retrying work. An alert fires to the operations team when a new dead-letter entry is created. Operators can manually re-queue a dead-letter message after the dependency is confirmed healthy, or mark it as resolved if the situation is handled by other means such as manual fulfillment or refund.

Dead-letter records are retained for 30 days and then archived.

7.7.3 Alternatives Considered

Separate message broker dead-letter queue. RabbitMQ supports dead-letter exchanges natively, and routing failed messages there is a common pattern. For this project, keeping the dead-letter record in the database gives operators a single interface for all integration work rather than requiring access to the broker management UI. It also makes dead-letter records part of the same backup and audit trail as orders.

Automatic retry without a dead-letter limit. Retrying indefinitely avoids requiring operator intervention but creates unbounded queue growth and makes it impossible to distinguish transient from permanent failures. Operations cannot act on a problem they cannot see.

7.7.4 Consequences

The dead-letter table is integration-critical infrastructure and must be monitored and backed up alongside the orders table. The operator intervention model introduces a human step in the recovery path, which is appropriate for permanent failures but requires clear UI affordances in the Support and Operations View. The 30-day retention policy must be aligned with the business's order dispute window.

Chapter 8

Risk and Validation Plan

8.1 Purpose

This chapter identifies the main architectural risks in evolving the current nopCommerce implementation into the Scenario C omnichannel commerce core described in the target architecture. The current implementation is treated as the baseline: nopCommerce already supports the customer-facing commerce flow, but the planned integration layer, adapters, retry metadata, reconciliation records, and operational visibility still need to be validated before they can be claimed as implemented architecture.

The validation plan therefore focuses on the pressure required by the assignment: surrounding operational systems may be delayed, stale, unavailable, or contradictory, while the commerce core must remain useful and understandable.

8.2 Main Architectural Risks

Severity is rated as High (H), Medium (M), or Low (L) based on the combination of likelihood and consequence if the risk is not mitigated.

Risk	Severity	Why it matters	Related scenarios
Surrounding systems unavailable while checkout is active	H	Warehouse or shipping failures must not cause paid orders to be lost or block later customers.	QAS 1, QAS 4
Inventory stale or contradictory across channels	H	Overselling or misleading availability information damages customer trust and creates fulfillment conflicts.	QAS 2, QAS 6
Retry or idempotency insufficient for async integration	H	Lost, duplicated, or unretried integration work is invisible and nonrecoverable without correct outbox and key design.	QAS 1, QAS 4; ADR 2, ADR 5
Degraded states not visible to operators	M	A technically resilient system is still operationally weak if support teams cannot explain or act on pending or failed state.	QAS 1, QAS 4; ADR 7
Integration work slows customer-facing path	M	Asynchronous work must not reenter the HTTP pipeline; any synchronous adapter call during checkout breaks QAS 3.	QAS 3
Future adapters bypass boundaries or share the nopCommerce database	M	Direct coupling makes new channels harder to add and weakens data ownership; easy to introduce accidentally during adapter development.	QAS 5; ADR 3
Dead-letter work silently grows without monitoring	M	Without alerts, failed integration work may accumulate unnoticed until an operator manually inspects the queue.	QAS 1, QAS 4; ADR 7

8.3 Validation Activities

Each activity includes a success criterion that defines what a passing result looks like. The first table explains the validation purpose; the second table defines the pass criteria and evidence to collect.

Validation activity	What it proves
Simulate warehouse timeout during order processing	Paid order is persisted locally, outbox record created, status marked pending, retry starts without operator action.
Simulate shipping provider unavailability after warehouse pack	Failed label requests are stored, orders enter awaiting-shipping state, labels are created automatically on recovery.
Simulate delayed stock update from POS or WMS	Stale stock is detected, marked, checkout revalidates, reconciliation record created if discrepancy found.
Verify idempotency under message replay	Replaying the same integration message does not create a duplicate fulfillment or label.
Verify operator visibility of integration states	Operators can distinguish normal, pending, degraded, failed, and dead-lettered work from the admin UI without database queries.
Order-burst responsiveness check	Customer checkout and order-status reads stay within QAS 3 response measures while async background work is queued.
Integration boundary review	No adapter reads or writes nopCommerce tables directly; all cross-boundary communication uses message contracts.

Validation activity	Success criterion	Evidence to collect
Simulate warehouse timeout during order processing	Order exists in DB; outbox record visible; admin shows "awaiting warehouse" within 5 s; first retry within 5 min.	Order record, outbox row, admin screenshot, retry log with timestamps
Simulate shipping provider unavailability after warehouse pack	Zero lost label requests; orders show awaiting-shipping; 95% of queued labels created within 10 min of provider recovery.	Retry table, admin view before and after recovery, label creation log
Simulate delayed stock update from POS or WMS	Stock marked stale within 30 s of threshold exceeded; checkout blocks or warns on pending reconciliation; reconciliation record created within 1 min.	Before/after stock views, `IsStale` flag log, reconciliation record
Verify idempotency under message replay	Exactly one warehouse acknowledgement per order after replaying the outbox message three times.	Warehouse adapter log, fulfillment count in admin, idempotency key deduplication record
Verify operator visibility of integration states	All five states visible through admin UI; no direct DB query needed.	Admin UI screenshots for each state, status transition log
Order-burst responsiveness check	Checkout p95 < 2 s; order-status p95 < 1 s; queue depth does not grow unboundedly during test.	Latency percentiles, queue depth over time, background worker throughput
Integration boundary review	Code review and network trace confirm no direct DB connection strings in adapter configs.	Code or config review notes, network trace or Wireshark capture

Chapter 9

Evolution Roadmap

The evolution roadmap describes how VerdeMart moves from the current nopCommerce baseline to the target omnichannel commerce core. The approach is incremental: each phase delivers a testable, runnable capability without taking the commerce core offline or rewriting checkout.

9.1 Implementation Order

Figure 9.1 shows the four phases in sequence and highlights what must coexist during each transition. Phase 1 is the prerequisite for everything else: no adapter can be built without a working outbox. Phases 3 and 4 both depend on Phase 2 having produced observable integration states, and Phase 4 draws on all earlier phases to build the operational visibility layer.

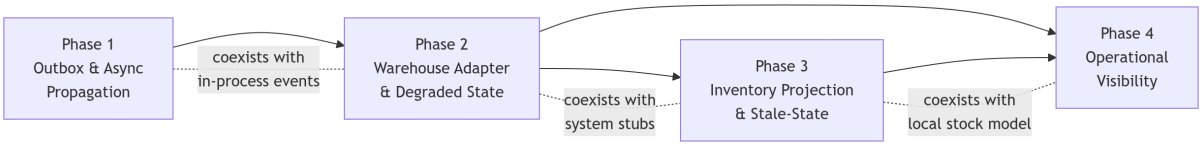

Figure 9.1: Implementation phases, ordering, and coexistence constraints.

9.1.1 Phase 1 — Outbox and Asynchronous Propagation

An outbox table is added to the nopCommerce database. Order placement writes an integration task in the same transaction as the order, and a scheduled task publishes unpublished records to the integration layer. This phase has no dependency on any external system; the publication destination can be a stub or log sink. What matters is proving the atomicity guarantee and the publishing lifecycle before any adapter is built on top.

During this phase, the outbox coexists with the existing in-process event bus. The `OrderPlacedEvent` continues to fire for internal reactions such as cache invalidation and email notifications. The outbox adds a durable external path alongside it, not in place of it.

9.1.2 Phase 2 — Warehouse Adapter and Degraded-State Handling

With a working outbox, the Warehouse Adapter is built and connected. This phase introduces the retry policy, idempotency key handling, and the integration state model: pending, sent, accepted, delayed, failed, and recovered. A warehouse simulator stands in for the real system and can be made slow or unavailable on demand, which is also how the mandatory pressure point from the scenario is demonstrated.

The shipping adapter follows the same pattern and can be introduced in parallel with this phase. System stubs coexist with the adapters throughout; real warehouse or carrier systems are not required to validate the architecture.

9.1.3 Phase 3 — Inventory Projection and Stale-State Visibility

The existing nopCommerce stock model is extended with source and freshness metadata. The Inventory Adapter receives stock updates from warehouse and store systems and updates the projection. When stock data is stale or conflicting, the model exposes a detectable state rather than silently serving an outdated number. Detected conflicts are recorded for reconciliation.

The local stock records that nopCommerce already manages are not replaced. The projection adds a visibility layer on top of them. Checkout continues to revalidate stock using the existing model; the projection is what operators and the storefront use to communicate freshness and discrepancy state.

9.1.4 Phase 4 — Operational Visibility

The operational read model aggregates the integration states produced in Phases 2 and 3 and exposes them through the nopCommerce admin area or an equivalent support-facing view. This phase introduces no new integration logic. Its purpose is to make earlier states visible and actionable: which orders are awaiting warehouse confirmation, which fulfillment requests have failed, and which inventory records are pending reconciliation.

Chapter 10

Feasibility Spike

10.1 Purpose

The feasibility spike focuses on the riskiest architectural assumption in this checkpoint: that nopCommerce can remain the commerce core while durable omnichannel integration is added around it. The spike therefore asks whether the current codebase has credible seams for the first target evolution step: recording an order, creating durable integration work, and processing that work outside the customer checkout path.

This chapter is intentionally scoped. It does not claim that the outbox, Integration Worker, RabbitMQ queues, warehouse adapter, retry policy, dead-letter queue, or support dashboard are already implemented. Those elements are part of the proposed target architecture. The spike checks whether they can be introduced without a rewrite, and whether the nopCommerce source already contains the specific seams that ADR 2 and the evolution roadmap depend on.

10.2 What Was Tested

The spike was conducted by reading the nopCommerce source directly against the classes named in the ADRs, rather than building a full runtime implementation. This was appropriate for the checkpoint because the main risk was architectural fit, not runtime behaviour: the question was whether nopCommerce already exposes a stable order-placement seam and a background execution mechanism credible enough to support the proposed asynchronous integration path.

Spike question	Inspection performed	Result
Can order creation be used as the trigger for external fulfillment work?	Inspected `OrderProcessingService` around the successful order-placement path.	Yes. The order is persisted, order items are created, notifications are sent, and an `OrderPlacedEvent` is published after order creation.
Is the current event mechanism sufficient for external integration?	Inspected `EventPublisher`.	No. Consumers run in process; exceptions are logged; there is no durable retry, replay, or dead-letter lifecycle.
Is there a background execution seam for retry or polling work?	Inspected `ScheduleTaskRunner` and scheduled-task behaviour.	Yes, with limits. Scheduled tasks can execute background work with locking and interval checks, but they need explicit integration state to be reliable.
Can existing inventory and shipment concepts support Scenario C projections?	Inspected warehouse inventory, stock history, product inventory adjustment, and shipment classes.	Partially. The baseline has stock, reserved quantity, stock history, tracking, shipped, delivered, and ready-for-pickup fields, but not source freshness, conflict flags, or integration-state metadata.

10.3 What Worked

The order-placement seam sits at exactly the right level. It fires after the database commit, which is where ADR 2's atomicity guarantee needs to be anchored: an outbox record can be written in the same transaction without touching the checkout flow or the payment handling path.

The locking and skip-if-running behaviour of `ScheduleTaskRunner` is exactly what an outbox publisher or retry task requires to avoid overlapping execution. A production Integration Worker would run as a separately deployable process, but a scheduled task is architecturally sufficient to prove the tactic without introducing a new deployment unit prematurely.

The gap in the existing stock and shipment records is not structural but additive. The commerce model already carries the right quantity and lifecycle fields; what is missing is integration metadata — a source identifier, freshness timestamp, stale flag, conflict flag, and retry count. Adding those fields extends the existing model rather than replacing it, which confirms the data ownership approach described in Chapter 6.

10.4 What Did Not Work Yet

The current implementation does not contain a transactional outbox, RabbitMQ integration, warehouse adapter, shipping adapter, retry scheduler, dead-letter table, or operator-facing integration dashboard. Therefore, the current codebase cannot yet demonstrate the mandatory Scenario C pressure point at runtime. If the warehouse is unavailable today, there is no durable fulfillment command that remains pending and retries automatically after recovery.

The in-process event mechanism identified in the spike as insufficient for durable external integration is not a deficiency in nopCommerce's design for its intended purpose. It is a structural gap relative to Scenario C, and it is precisely what ADR 2 addresses. The spike therefore confirms the need for the outbox rather than raising doubt about the approach.

10.5 Evidence

The three most critical findings are supported by direct source references from the nopCommerce repository.

Insertion point in `OrderProcessingService`. The `PlaceOrderAsync` method in `Nop.Services/Orders/O` ends its successful path with the following sequence (lines 1608–1621):

```
//notifications
await SendNotificationsAndSaveNotesAsync(order);
//reset checkout data
await _customerService.ResetCheckoutDataAsync(...);
await _customerActivityService.InsertActivityAsync(...);
//raise event
await _eventPublisher.PublishAsync(new OrderPlacedEvent(order));
//check order status
await CheckOrderStatusAsync(order);
```

The order and its line items are already committed to the database before this point. An outbox record inserted here — in the same transaction scope — would satisfy the atomicity guarantee required by ADR 2 without touching the checkout flow or the payment handling path.

Durability gap in `EventPublisher`. The `PublishAsync` method in `Nop.Services/Events/EventPublishe` resolves registered consumers and dispatches each one in sequence:

```
foreach (var consumer in consumers)
{
    try { await consumer.HandleEventAsync(@event); }
    catch (Exception exception)
    {
        await logger.ErrorAsync(exception.Message, exception);
    }
}
```

Exceptions are caught, logged, and swallowed. No failed work is persisted as a retryable record. This is the structural gap that ADR 2 exists to close.

Locking behaviour in `ScheduleTaskRunner`. The `ExecuteAsync` method in `Nop.Services/ScheduleTask` skips execution if a previous run is still in progress and otherwise acquires a distributed lock before executing:

```
if (IsTaskAlreadyRunning(scheduleTask))
    return;
...
await _locker.PerformActionWithLockAsync(
    scheduleTask.Type, expiration,
    () => PerformTaskAsync(scheduleTask));
```

This guarantees that an outbox publisher or retry task does not overlap with a previous execution, which is the minimum safety property required for the outbox publishing loop.

The remaining findings — the inventory and shipment fields available on `ProductWarehouseInventory`, `StockQuantityHistory`, and `Shipment` — were confirmed by inspecting the corresponding entity classes. They confirm that the extension model described in Chapter 6 is additive rather than structural: the commerce records are already there, and only integration metadata fields are missing.

10.6 Smallest Implementation Slice for the Final Demo

If implementation time is limited, the final demo should not attempt the whole target architecture. The smallest useful slice is:

1. Add an `OmnichannelIntegrationTask` or outbox table with order ID, message type, payload, status, retry count, next attempt time, correlation ID, idempotency key, and last error.
2. Create one outbox record when an order is accepted. The record should represent `FulfillmentRequested`.
3. Add a scheduled task that reads pending records and calls a warehouse simulator or stub endpoint.
4. Make the simulator return success, timeout, and failure modes through configuration.
5. On failure, update the task to retrying or failed with a visible error. On success, mark it accepted or recovered.
6. Add a minimal admin or log-based view showing pending, retrying, failed, and recovered states for one order.

This slice directly demonstrates QA Scenario 1: checkout records the order, warehouse failure does not block the customer path, the fulfillment request is retained as an observable record, and recovery is visible after the dependency becomes available again. Shipping, inventory conflict handling, and dead-letter intervention can remain documented target behaviour if implementation time is limited, because the spike has confirmed that the seams for all of them already exist in the codebase.